AGENTIC AI WORKSHOP · DAY 3 OF 10

Framework foundations: LangChain & LCEL

Port the Day 2 agent to LangChain: components, LCEL pipelines, typed (structured) output and model routing.

You build: **Framework** · about 2.3 h of material

Contents

1. From API caller to systems engineer — Why this matters · 5 min
2. What LangChain replaces — Theory · 8 min
3. LCEL: build pipelines with | — Theory · 10 min
4. Lab 1: Port Meridian to LangChain — Lab · beginner · 40 min
5. Structured output: objects, not text — Theory · 8 min
6. Lab 2: Triage merchant emails into typed tickets — Lab · intermediate · 25 min
7. Model routing — Theory · 8 min
8. Lab 3: Route between Claude and a local model — Lab · intermediate · 25 min
9. Check yourself — Quiz · 5 min
10. Day 3 recap — Recap · 3 min
11. Quiz answer key

WHY THIS MATTERS 5 MIN

From API caller to systems engineer

Yesterday you wrote ~100 lines and it worked. So why add a framework?

An API caller worries about…	A systems engineer worries about…
getting one answer back	50 agents, 6 models, 3 teams
one provider's message format	swapping providers without rewrites
a `while` loop	retries, streaming, async, batching, timeouts
state in a Python list	persistence and human approval (Day 4)

A framework gives you **one common interface**, standard message and tool types, and an ecosystem (tracing, vector stores, checkpointers) that plugs in without glue code. The cost is abstraction and version churn, which is why you hand-built the loop first.

By the end of today you can

- Point to exactly which Day 2 lines LangChain replaces
- Compose pipelines with LCEL (prompt | model | parser)
- Get typed pydantic objects back instead of parsing text
- Route each task to Claude or a local model through one interface

THEORY 8 MIN

What LangChain replaces

Every line on the left has a line on the right.

Day 2 (by hand)	Day 3 (LangChain)	Layer
`client.messages.create(model=..., tools=TOOLS)`	`ChatOpenRouter(model="anthropic/claude-opus-5.5")`	chat model
pydantic model → `model_json_schema()` → `TOOLS` list	`@tool def get_fx_rate(base, quote)`	tool (schema from type hints)
`run_tool()` + `is_error`	`ToolNode` inside `create_agent`	tool execution
`for turn in range(MAX_TURNS)` loop	`create_agent(...)` + `recursion_limit`	agent loop
`messages.append(...)`	`{"messages": [...]}` state	state
JSON prompt + `model_validate`	`model.with_structured_output(Plan)`	structured output

Key points

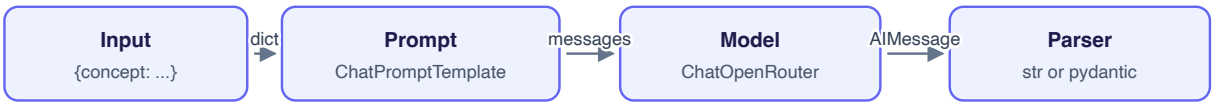
- create_agent IS the Day 2 loop, packaged as a graph. On Day 4 you'll open that graph up.
- Keep business rules in your code, not in the framework.

THEORY 10 MIN

LCEL: build pipelines with |

Anything with .invoke() is a Runnable. The | operator joins Runnables into a new Runnable.

prompt | model | parser: what flows through each pipe



1. A dict of variables fills the prompt template.
2. The prompt becomes chat messages for the model.
3. The model's AIMessage is parsed into a string, or a typed pydantic object.
4. The whole chain is itself a Runnable: .invoke(), .batch() (parallel), .stream() and async come free.

Chains vs agents: a chain is a **path you designed**; an agent **chooses** its path. Classifying 10,000 support emails is a chain: cheaper, faster, testable. Investigating why a merchant is over its limit is an agent.

LAB BEGINNER 40 MIN

Lab 1: Port Meridian to LangChain

Same agent, two ways: first the loop with bind_tools (maps 1:1 to Day 2), then create_agent (the loop disappears).

Goal. Meridian answers the Day 2 questions using LangChain components.

1. Create the project kit

From today, work in one project folder that grows each day. Create these three files (plus your `.env` with `OPENROUTER_API_KEY`).

requirements.txt

```
anthropic>=1.11,<2
python-dotenv>=1.2,<2
pydantic>=2.13,<3
langchain>=1.4,<2
langchain-openrouter>=0.2.9,<0.3
langchain-ollama>=1.1,<2
```

2. data.py: the merchants and FX rates

data.py

```
"""data.py: Northstar Pay mock data (fictional payment processor). Deterministic so answers are
reproducible.

Answer key (verdict rules: >100% OVER_LIMIT, >90% NEAR_LIMIT, else OK):
  ACME: 6.2M EUR x 1.10 + 3.1M USD = 9.92M USD vs 9.5M USD  -> OVER_LIMIT (104.4%)
  BOLT: 1.5M USD / 1.30 + 2.0M GBP = 3.154M GBP vs 5M GBP   -> OK (63.1%)
  CRUX: 650M INR + 2M USD x 85 = 820M INR vs 800M INR       -> OVER_LIMIT (102.5%), chargeback
watchlist
"""

MERCHANTS = {
    "ACME": {
        "name": "ACME Electronics Online (fictional)",
        "mcc": "5732",
        "channel": "e-commerce",
        "risk_tier": "Medium",
        "processing_limit": {"amount": 9_500_000, "currency": "USD", "period": "monthly"},
        "settlement_currency": "USD",
        "chargeback_ratio_pct": 0.6,
        "rolling_reserve_pct": 5,
        "mtd_volume": [
            {"batch_id": "B-1001", "method": "card_cnp", "amount": 4_200_000, "currency": "EUR"},
            {"batch_id": "B-1002", "method": "card_cnp", "amount": 3_100_000, "currency": "USD"},
            {"batch_id": "B-1003", "method": "wallet", "amount": 2_000_000, "currency": "EUR"},
        ],
        "kyb_notes": "Annual KYB review due 2026-11-30. No adverse media.",
    },
    "BOLT": {
        "name": "Bolt Travel Ltd (fictional)",
        "mcc": "4722",
        "channel": "e-commerce",
        "risk_tier": "Low",
        "processing_limit": {"amount": 5_000_000, "currency": "GBP", "period": "monthly"},
        "settlement_currency": "GBP",
```

```
        "chargeback_ratio_pct": 0.3,
        "rolling_reserve_pct": 0,
        "mtd_volume": [
            {"batch_id": "B-2001", "method": "card_cnp", "amount": 1_500_000, "currency": "USD"},
            {"batch_id": "B-2002", "method": "card_cnp", "amount": 2_000_000, "currency": "GBP"},
        ],
        "kyb_notes": "Enhanced due diligence completed 2026-06.",
    },
    "CRUX": {
        "name": "Crux Gaming Pvt Ltd (fictional)",
        "mcc": "7995",
        "channel": "e-commerce",
        "risk_tier": "High",
        "processing_limit": {"amount": 800_000_000, "currency": "INR", "period": "monthly"},
        "settlement_currency": "INR",
        "chargeback_ratio_pct": 1.2,
        "rolling_reserve_pct": 10,
        "mtd_volume": [
            {"batch_id": "B-3001", "method": "upi", "amount": 650_000_000, "currency": "INR"},
            {"batch_id": "B-3002", "method": "card_cnp", "amount": 2_000_000, "currency": "USD"},
        ],
        "kyb_notes": "Watchlist: chargeback ratio above scheme monitoring threshold (0.9%) since 2026-
09.",
    },
}

# Spot rates: 1 unit of base -> quote.
FX = {
    ("EUR", "USD"): 1.10,
    ("GBP", "USD"): 1.30,
    ("USD", "INR"): 85.0,
    ("USD", "JPY"): 150.0,
    ("USD", "CHF"): 0.88,
}

AS_OF = "2026-10-02T16:00Z"


def fx_rate(base: str, quote: str) -> float:
    """Direct, inverted, or via-USD cross rate. Raises LookupError if impossible."""
    if base == quote:
        return 1.0
    if (base, quote) in FX:
        return FX[(base, quote)]
    if (quote, base) in FX:
        return round(1 / FX[(quote, base)], 8)
    if base != "USD" and quote != "USD":
        return round(fx_rate(base, "USD") * fx_rate("USD", quote), 8)
    raise LookupError(f"no rate for {base}/{quote}")


def expected_verdict(merchant_id: str) -> dict:
    """Ground truth used by tests and evals (Day 8)."""
    m = MERCHANTS[merchant_id]
    cur = m["processing_limit"]["currency"]
    total = sum(b["amount"] * fx_rate(b["currency"], cur) for b in m["mtd_volume"])
    pct = total / m["processing_limit"]["amount"] * 100
    verdict = "OVER_LIMIT" if pct > 100 else "NEAR_LIMIT" if pct > 90 else "OK"
    return {"merchant_id": merchant_id, "mtd_volume": round(total, 2), "currency": cur,
            "utilisation_pct": round(pct, 1), "verdict": verdict}


if __name__ == "__main__":
    for mid in MERCHANTS:
        print(expected_verdict(mid))
```

3. llm.py: Claude via OpenRouter, for raw SDK and LangChain

llm.py

```
"""llm.py (Day 3+): Claude via OpenRouter, for raw-SDK code AND for LangChain."""
import os

import anthropic
from dotenv import load_dotenv
from langchain_openrouter import ChatOpenRouter

load_dotenv()  # reads OPENROUTER_API_KEY from .env

MODEL = os.getenv("MODEL", "anthropic/claude-opus-5.5")
MODEL_FAST = os.getenv("MODEL_FAST", "anthropic/claude-haiku-4.5")

# Raw Anthropic SDK (as on Day 2)
client = anthropic.Anthropic(base_url="https://openrouter.ai/api",
auth_token=os.environ["OPENROUTER_API_KEY"])


def chat_model(model: str = MODEL, **kwargs) -> ChatOpenRouter:
    """A LangChain chat model for Claude via OpenRouter (reads OPENROUTER_API_KEY)."""
    kwargs.setdefault("max_tokens", 8000)
    return ChatOpenRouter(model=model, **kwargs)
```

4. tools_lc.py: tools as @tool functions

Print `get_fx_rate.args_schema.model_json_schema()` : it's the schema you wrote by hand yesterday.

tools_lc.py

```
# tools_lc.py: Meridian's tools as LangChain @tool functions (schema comes from type hints +
docstring)
import ast
import operator
from typing import Literal

from langchain_core.tools import tool

from data import MERCHANTS, fx_rate

Currency = Literal["USD", "EUR", "GBP", "INR", "JPY", "CHF"]
OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul, ast.Div:
operator.truediv}


def _calc(node):
    if isinstance(node, ast.Constant):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in OPS:
        return OPS[type(node.op)](_calc(node.left), _calc(node.right))
    raise ValueError("only numbers and + - * / are allowed")


@tool
def calculator(expression: str) -> float:
    """Evaluate an arithmetic expression (+ - * / parentheses). Use for ALL arithmetic."""
    return round(_calc(ast.parse(expression, mode="eval").body), 4)


@tool
def lookup_merchant(merchant_id: str) -> dict:
```

```
    """Look up a merchant file: monthly processing limit (amount + currency), risk tier, chargeback
ratio
    and this month's settlement batches (amount + currency)."""
    m = MERCHANTS.get(merchant_id.strip().upper())
    if m is None:
        raise ValueError(f"unknown merchant; valid ids: {sorted(MERCHANTS)}")
    return m


@tool
def get_fx_rate(base: Currency, quote: Currency) -> float:
    """Spot FX rate converting 1 unit of `base` into `quote`. Use for ANY currency conversion; never
guess rates."""
    return fx_rate(base, quote)


TOOLS = [calculator, lookup_merchant, get_fx_rate]
```

5. The loop with bind_tools

Annotate each line with the Day 2 line it replaced.

agent_bind_tools.py

```
# agent_bind_tools.py
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

from langchain_openrouter import ChatOpenRouter      # Claude via OpenRouter
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage
from tools_lc import TOOLS

model = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=16000).bind_tools(TOOLS)
by_name = {t.name: t for t in TOOLS}

SYSTEM = ("You are Meridian, a merchant-risk & payments ops analyst at Northstar Pay. Use only tools.
"
          "Convert every amount into the limit's currency. Finish with OVER_LIMIT / NEAR_LIMIT / OK +
utilisation %.")

def run(question: str, max_turns: int = 10) -> str:
    messages = [SystemMessage(SYSTEM), HumanMessage(question)]
    for _ in range(max_turns):
        ai = model.invoke(messages)                    # ← was client.messages.create
        messages.append(ai)
        if not ai.tool_calls:                          # ← was stop_reason == "end_turn"
            return ai.text                             # .text property on langchain-core 1.x
messages
        for call in ai.tool_calls:                     # ← was iterating tool_use blocks
            try:
                out = by_name[call["name"]].invoke(call["args"])
                messages.append(ToolMessage(str(out), tool_call_id=call["id"]))
            except Exception as e:
                messages.append(ToolMessage(f"error: {e}", tool_call_id=call["id"], status="error"))
    return "[stopped: max_turns]"
```

6. create_agent: the loop disappears

agent_create.py

```
# agent_create.py
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env
```

```python
from langchain.agents import create_agent
from tools_lc import TOOLS

meridian = create_agent(
    model="openrouter:anthropic/claude-opus-5.5",      # or pass a ChatOpenRouter instance
    tools=TOOLS,
    system_prompt=("You are Meridian, a merchant-risk & payments ops analyst at Northstar Pay. Use
only tools. "
                   "Convert every amount into the limit's currency. Finish with OVER_LIMIT /
NEAR_LIMIT / OK + utilisation %."),
)

result = meridian.invoke(
    {"messages": [{"role": "user", "content": "Is ACME over its processing limit?"}]},
    config={"recursion_limit": 20},          # ← was MAX_TURNS
)
for m in result["messages"]:
    m.pretty_print()
```

7. Run both and compare

Same answers as Day 2? ACME → OVER_LIMIT (104.4%).

RUN

```
python agent_bind_tools.py
python agent_create.py
```

Expected result. Both versions call lookup_merchant → get_fx_rate → calculator and answer **OVER_LIMIT** for ACME.

THEORY 8 MIN

Structured output: objects, not text

Downstream code (routing, databases, dashboards) needs typed data, not prose.

Approach	Reliability	Use it
Regex on free text	❌ breaks when the model rephrases	never in new code
Format instructions + output parser	⚠️ medium	models without native tool/JSON support
`model.with_structured_output(MyModel)`	✅ high	the default
`create_agent(..., response_format=MyModel)`	✅ high	when an **agent** must return a typed answer

LangChain uses the provider's native mechanism (tool calling or JSON schema) under the hood. It's the same idea as Day 2's "JSON + pydantic validate", done for you.

Key points

- Pydantic is the contract between the probabilistic part (the model) and the deterministic part (your code).

LAB INTERMEDIATE 25 MIN

Lab 2: Triage merchant emails into typed tickets

Goal. A chain (not an agent) that turns free-text merchant emails into TriageTicket objects.

1. The input: three merchant support emails

`emails`

```
EMAILS = [
    "Hi, this is ACME Electronics. Settlement batch B-1003 (2M EUR) hasn't reached our account and
it's been 3 days. "
    "Our suppliers are waiting. Please fix by EOD.",
    "URGENT from CRUX Gaming: we got 340 chargebacks overnight on card payments, about 2M USD total.
We think it's a fraud "
    "attack. Can you block the BINs and help us fight the disputes?",
    "BOLT Travel here. Our checkout returns HTTP 429 on refunds during peak hours. Not urgent, but can
we get a higher refund rate limit?",
]
```

2. Build the pipeline: prompt | model.with_structured_output(TriageTicket)

Notice `.batch()`: all three emails run in parallel for free.

`triage_pipeline.py`

```
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

from typing import Literal
from pydantic import BaseModel, Field
from langchain_core.prompts import ChatPromptTemplate
from langchain_openrouter import ChatOpenRouter      # Claude via OpenRouter

class TriageTicket(BaseModel):
    """Triage ticket for a merchant support email."""
    merchant_id: str | None = Field(description="ACME, BOLT, CRUX… or null if not identifiable")
    batch_id: str | None = Field(description="Settlement batch id like B-1003, if mentioned")
    category: Literal["payout_delay", "chargeback_dispute", "fraud_suspected", "refund_issue",
"api_integration", "other"]
    severity: Literal["P1", "P2", "P3"] = Field(description="P1 = money at risk or merchant cannot
trade today")
    amount: float | None = Field(description="Amount in major units, if stated")
    currency: str | None
    next_action: str = Field(description="One imperative sentence for the support agent")

prompt = ChatPromptTemplate.from_messages([
    ("system", "You triage merchant support emails for Northstar Pay, a payment processor. "
               "Suspected fraud attacks and chargeback spikes are always P1. Payout delays over 2 days
are P1. "
               "API questions are P3 unless payments are failing. Never include card numbers in any
field."),
    ("human", "{email}"),
])

model = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=4000)
triage = prompt | model.with_structured_output(TriageTicket)

tickets = triage.batch([{"email": e} for e in EMAILS])   # parallel, free from LCEL
for t in tickets:
    print(t.model_dump_json(indent=2))
```

3. **Run it and check the severities**

   The CRUX chargeback spike should be **P1** and `fraud_suspected`; BOLT's API question **P3**.

4. **Extend it**

   Look up the merchant's risk tier from `data.py` before the prompt (`RunnablePassthrough.assign`). Rule: High-tier merchants bump severity one level.

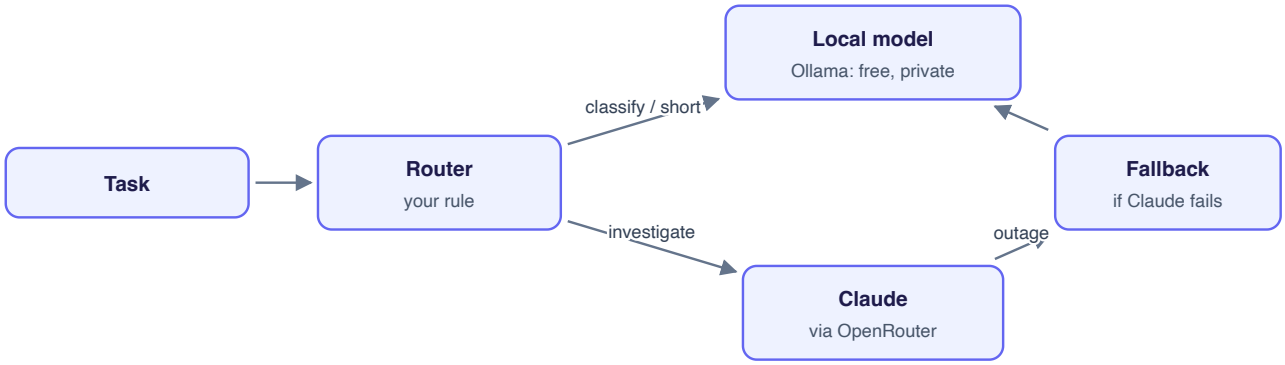
Expected result. Three JSON tickets: ACME `payout_delay` P1, CRUX `fraud_suspected` P1, BOLT `api_integration` P3.

THEORY 8 MIN

Model routing

Same interface, different model per call. Route each task to the cheapest model that's good enough.

Tiered routing with a fallback



1. A readable rule in code decides the route. Routing is business logic: keep it explicit and logged.
2. Narrow tasks (classify tone, short summaries) go to a local model: free and private.
3. Multi-step reasoning with tools goes to Claude.
4. .with_fallbacks([cheap]): if Claude is down, the local model answers, with a degraded-mode warning.

Key points

- Small local models are fine for narrow tasks and weak at multi-step tool use.
- Routing is a MEASURED decision: Day 8 evals tell you when a cheaper model is good enough.

LAB INTERMEDIATE 25 MIN

Lab 3: Route between Claude and a local model

Goal. One interface, two models: a tiered router and a fallback.

1. Pull a local model

RUN

```
ollama pull qwen3:8b
pip install langchain-ollama
```

2. Two models, one interface

`usage_metadata` has the same shape for both: that matters for the Day 8 cost ledger.

routing.py: two models

```
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

from langchain_openrouter import ChatOpenRouter      # Claude via OpenRouter
from langchain_ollama import ChatOllama          # NOT langchain_community (deprecated)

smart = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=8000)
cheap = ChatOllama(model="qwen3:8b")

q = "In one sentence, what is a chargeback, and why does it cost the acquirer money?"
for name, m in [("claude", smart), ("ollama", cheap)]:
    r = m.invoke(q)
    print(name, "→", r.text, "\n   usage:", r.usage_metadata)
```

3. A tiered router

routing.py: router

```
from langchain_core.runnables import RunnableLambda

def route(inp: dict):
    # Simple, explicit rule. Teach that routing rules are business logic: keep them readable and
logged.
    return cheap if inp["task"] in {"classify", "summarise_short"} else smart

router = RunnableLambda(lambda inp: route(inp).invoke(inp["prompt"]))

print(router.invoke({"task": "classify", "prompt": "Classify tone (neutral/negative): 'BOLT says their
payout is late again.'"}).text)
print(router.invoke({"task": "investigate", "prompt": "Explain how a 2M EUR position could push ACME
over a 9.5M USD limit."}).text)
```

4. A fallback for outages

We simulate an outage with an invalid key.

routing.py: fallback

```
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

resilient = smart.with_fallbacks([cheap])

# Simulate an outage: use a bad API key for the Claude client only
```

```
broken_smart = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=8000, api_key="sk-or-
invalid")
resilient_demo = broken_smart.with_fallbacks([cheap])
print(resilient_demo.invoke("Define merchant discount rate (MDR) in one line.").text)    # answered by
Ollama
```

5. **Stretch: run the Lab 1 agent on the local model**

   Does it still convert EUR before comparing? Record correct/incorrect for ACME, BOLT and CRUX.

Expected result. Classification answered by Ollama, the investigation by Claude, and the fallback answers even with a broken key.

QUIZ 5 MIN

Check yourself

1. **What does prompt | model | parser produce?**
   A. A string
   B. A new Runnable (a RunnableSequence)
   C. A tool
   D. An agent

2. **Chain or agent: classify 10,000 support emails into categories?**
   A. Agent
   B. Chain

3. **Why use with_structured_output instead of regex?**
   A. It's shorter
   B. It uses native constrained output and returns typed objects; it doesn't break when the model rephrases
   C. Regex is deprecated
   D. It's free

4. **Which Day 2 construct became recursion_limit?**
   A. TOOLS
   B. MAX_TURNS
   C. run_tool
   D. SYSTEM

5. **Name a good reason to route a task to a local model.**
   A. It's always more accurate
   B. Data residency, cost on high-volume narrow tasks
   C. It has more tools
   D. It never needs evaluation

RECAP 3 MIN

Day 3 recap

- A framework gives a common interface; you still own the business rules.
- LCEL: everything is a Runnable; | composes them.
- Chains for fixed paths, agents for open-ended ones.
- with_structured_output returns typed pydantic objects.
- Route by task: local for narrow, Claude for reasoning, with a fallback.

Next: Day 4: open up create_agent's hidden graph. Meridian gets branching, memory, and a human approval gate before it raises a limit.

Quiz answer key

1. **B. A new Runnable (a RunnableSequence)**
   Composition gives you another Runnable, with invoke/batch/stream for free.
2. **B. Chain**
   The path is fixed and known, so a chain is cheaper, faster and testable.
3. **B. It uses native constrained output and returns typed objects; it doesn't break when the model rephrases**
   Typed objects are the contract between the model and your code.
4. **B. MAX_TURNS**
   Both stop a runaway loop.
5. **B. Data residency, cost on high-volume narrow tasks**
   Local models keep data on-prem and cost nothing per call, but they're weaker at reasoning.